Question (i)

In [17]:
```
import numpy as np
import matplotlib.pyplot as plt
```

In [18]:
```
N =512
dt=4 / N
t1=np.arange(0,128) * dt
t2=np.arange(128,384) * dt
t3 =np.arange(384,512) * dt
s1 = np.exp(t1-1)
s2 = (t2 -2)**2
s3 = np.exp(3-t3)
s = np.concatenate([s1, s2,s3]).reshape(-1,1)
print('Samplecount:', s.shape[0])
print('First 5 samples:', s[:5,0])
```

```
Samplecount: 512
First 5 samples: [0.36787944 0.37076476 0.3736727  0.37660345 0.37955719]
```

Question (ii)

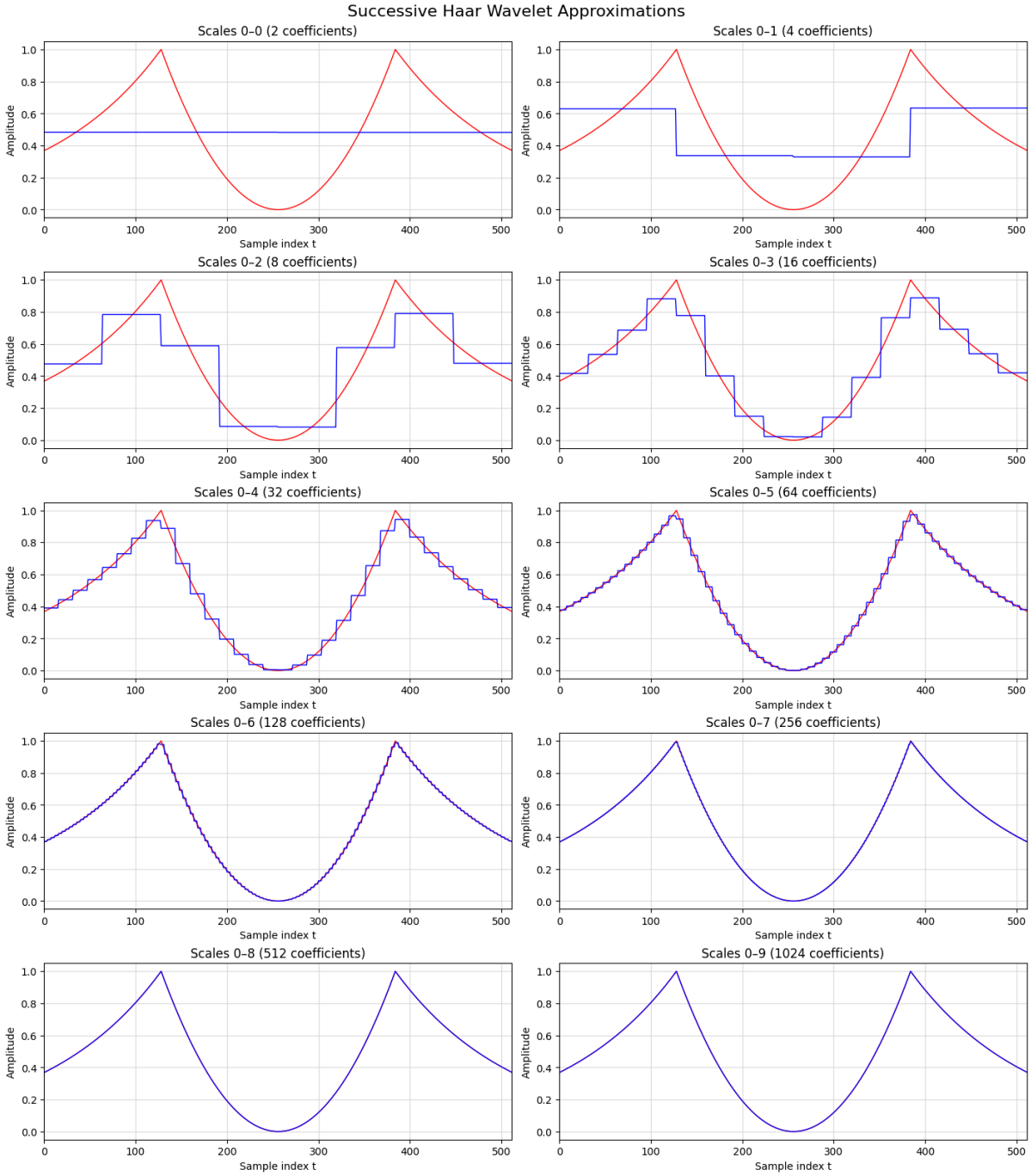
In [19]:
```
def normalized_haar_matrix(n):
    if n ⩽ 0 or (n & (n - 1)) ≠ 0:
        raise ValueError('n must be a positive power of two')
    columns = [np.ones(n) / np.sqrt(n)]
    for scale in range(int(np.log2(n))):
        block = n // (2**scale)
        half = block // 2
        for offset in range(0,n,block):
            wavelet = np.zeros(n)
            wavelet[offset:offset + half] = 1/np.sqrt(block)
            wavelet[offset +half:offset + block]= -1 /np.sqrt(block)
            columns.append(wavelet)
    return np.column_stack(columns)
V=normalized_haar_matrix(N)
c = V.T@ s
reconstructed = V@ c
reconstruction_error =np.linalg.norm(s - reconstructed)
print('V shape:',V.shape)
print('Reconstruction error:',f'{reconstruction_error:.12g}')
assert np.allclose(reconstructed,s)
```

```
V shape: (512, 512)
Reconstruction error: 2.00820748518e-15
```

Question (iii)

In [20]:

```
t = np.arange(512)
fig, axes = plt.subplots(5, 2, figsize=(14, 16), constrained_layout=True)
for k, ax in enumerate(axes.flat):
    n_coeffs = 2**(k + 1)
    yk = V[:, :n_coeffs] @ c[:n_coeffs]
    ax.plot(t, s[:,0],'r-',linewidth=1,label='s')
    ax.plot(t, yk[:,0],'b-',linewidth=1,label='Approximation')
    ax.set_title(f'Scales 0-{k} ({n_coeffs} coefficients)')
    ax.set_xlabel('Sample index t')
    ax.set_ylabel('Amplitude')
    ax.set_xlim(0,511)
    ax.grid(True,alpha=.4)
fig.suptitle('Successive Haar Wavelet Approximations', fontsize=16)
plt.savefig('Lab7_Haar_Scales.png', dpi=180, bbox_inches='tight')
plt.show()
```



Question (iv)

In [21]:
```
n_coeffs = 2**(7 + 1)
y = V[:, :n_coeffs] @ c[:n_coeffs]
relative_rmse = np.linalg.norm(s - y) / np.linalg.norm(s)
print(f'Relative root mean square error = {relative_rmse:.12f}')
print(f'Percentage ={100 *relative_rmse:.4f}%')
```

```
Relative root mean square error = 0.006527228121
Percentage = 0.6527%
```

Graph — Answer

In [22]:
```
plt.figure(figsize=(12, 5))
plt.plot(t, s[:,0], 'r-', linewidth=1.5, label='Original s')
plt.plot(t, y[:,0], 'b-', linewidth=1.1, label='Approximation y (scales 0-7)')
plt.title('Original Signal and Haar Approximation (Scales 0-7)')
plt.xlabel('Sample index t (0-511)')
plt.ylabel('Amplitude')
plt.xlim(0, 511)
plt.grid(True, alpha=.4)
plt.legend(loc='best')
plt.savefig('Lab7_Comparison.png', dpi=200, bbox_inches='tight')
plt.show()
```

